SLIIT
Discover Your Future

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 **MLOPs Report**

Group ID: 25-26J-103

Project Title: E-Learning Platform for hearing impaired Students

1. IT22546852 : AI-DRIVEN REAL-TIME SLSL E-LEARNING WITH GESTURE CORRECTION & 3D AVATAR STYLES
2. IT22143754: AI-POWERED SINHALA LIP-READING ASSISTANT WITH 3D AVATAR FEEDBACK
3. IT22255488: EMOTION AWARE MUSIC FOR HEARING IMPAIRED STUDENTS
4. IT22561466: EMOTION MONITORING WITH REAL-TIME FEEDBACK AND STAGEBASED LEARNING OPTIMIZATION

		IT22546852	IT22143754	IT22255488	IT22561466
Data Pipeline	**Data Sources:**	• SLSL dataset from kaggel • Custom Sri Lankan Sign Language (SSL) gesture dataset (images/videos collected from hearing-impaired individuals and online SSL resources) • Annotated gesture samples labeled according to SSL vocabulary	• custom Sinhala lip-reading datase (Participants perform predefined words and sentences. Each word and sentence is recorded using a standard webcam setup ensuring consistent background, lighting, and camera positioning.	• Audio files (WAV, MP3, FLAC, OGG, M4A) labeled with valence & arousal values on a 1–9 scale sourced from music emotion datasets	• Custom facial expression and attention monitoring dataset collected using webcam video recordings • Public face detection and emotion recognition datasets (e.g., FER2013 / MediaPipe sample facial datasets)

					• Custom dataset containing student face images and short video clips for gaze detection and screen attention monitoring • Annotated labels for: ✓ focused ✓ distracted ✓ confused ✓ absent ✓ looking at screen ✓ looking away
	Data Preprocessing:	• Frame extraction from gesture videos • Hand landmark detection using MediaPipe • Normalization of hand coordinates (scaling, centering) • Data augmentation (rotation, flipping, noise injection) to improve robustness	• Face detection • Lip region extraction (ROI) • Landmark detection (MediaPipe Face Mesh) • Frame normalization (resize, scaling)	• Resample audio to 44,100 Hz mono • Segment into 5-second chunks (220,500 samples); zero-pad partials > 2.5 s • Compute 128-band mel-spectrogram (n_fft=2048, hop_length=512) • Convert to dB scale via librosa.power_to_db() • MinMax-normalise valence & arousal labels (scikit-learn) Trained model serialised	• **Face detection using OpenCV Haar Cascade / MediaPipe Face Detection** • Facial landmark extraction using **MediaPipe Face Mesh** • Eye region detection for gaze tracking

		• Sequence padding/truncation for consistent input length	• Sequence creation for temporal frames	as best_cnn_model.keras (Keras format); loaded at server start-up for inference. Training label statistics (min/max) stored for denormalisation.	• Head pose estimation for screen attention monitoring • Face alignment and normalization • Image resizing and pixel scaling • Frame extraction from webcam video stream • Sequence generation for temporal attention analysis • Data augmentation: ✓ brightness adjustment ✓ rotation ✓ slight zoom ✓ horizontal shift • Label encoding for attention states
	Data storage and versioning	• Dataset stored in structured directories	• Dataset stored as: • Frame sequences	• Trained model serialised as best_cnn_model.keras (Keras format); loaded at server start-	• Face image datasets stored in **train /**

		(train/validation/test split) • Version control using Git • Preprocessed data stored as NumPy for faster training	• Labels (word + phoneme) • Version control is maintained using Git-based tracking	up for inference. Training label statistics (min/max) stored for denormalisation.	**validation / test directory structure** • Preprocessed facial frames stored as **NumPy arrays (.npy)** for efficient model training • Video frame sequences stored with timestamp-based naming • Version control maintained using **Git** • Model experiment tracking using version-based folders • Dataset versions maintained for reproducibility
Model Development	**Model Selection**	• CNN + LSTM hybrid architecture • CNN → spatial feature extraction (hand shape)	• CNN → extracts lip shape • LSTM → captures temporal motion	• CNN Convolutional Neural Network mapping mel-spectrograms to Russell's Circumplex Model of Affect (2-D valence/arousal space). Architecture includes batch	• **CNN architecture** for face feature extraction • **CNN + LSTM hybrid model** for temporal

		• LSTM → temporal sequence learning (gesture motion)		normalisation and dropout layers.	attention monitoring • CNN extracts: ✓ facial features ✓ eye region features ✓ expression patterns • LSTM captures: ✓ gaze movement sequence ✓ head movement over time ✓ temporal attention changes
	Model Training	• Loss Function: Categorical Cross-Entropy • Optimizer: Adam • Evaluation Metrics: Accuracy, Precision, Recall • Training done using TensorFlow • Early stopping and dropout used to prevent overfitting • Model Integration	Training pipeline includes: • Input: Lip frame sequences • Output: Word / viseme class Config: • Loss: Categorical Cross-Entropy • Optimizer: Adam • Metrics: Accuracy Training pipeline automated Hyperparameters tracked Model versions saved	• Framework: TensorFlow / Keras Loss: MSE | Optimiser: Adam Labels: MinMax-normalised to [0,1]; denormalised post-prediction Valence range: [1.6, 8.4] · Arousal range: [1.8, 8.1] Evaluation: Multi-run variance testing for prediction consistency	• Loss Function: **Categorical Cross-Entropy** • Optimizer: **Adam** • Metrics: ✓ Accuracy ✓ Precision ✓ Recall ✓ F1-score • Early stopping used to reduce overfitting

					• Dropout layers added for regularization • Training performed using **TensorFlow / Keras** • Hyperparameters tracked: ✓ learning rate ✓ batch size ✓ epochs ✓ dropout rate
Model Integration	**Tools used**	• Frontend: React (for interactive learning UI) • Backend: Flask REST API • ML Framework: TensorFlow • Computer Vision: MediaPipe (hand tracking) + OpenCV • Real-time Processing: WebRTC / browser webcam integration Model Deployment	• Frontend: React (interactive learning interface) • Backend: Flask REST API • Machine Learning: TensorFlow • Computer Vision: MediaPipe + OpenCV	• **TensorFlow ≥ 2.20 / Keras** — inference • **librosa ≥ 0.10** — mel-spectrogram & audio resampling • **NumPy ≥ 1.26** — array operations • **scikit-learn ≥ 1.2** — MinMaxScaler normalization • **FastAPI ≥ 0.104 + uvicorn** — REST endpoint /predict • **soundfile ≥ 0.12** — audio I/O • **python-multipart ≥ 0.0.6** — WAV blob upload handling	• Frontend: **React** • Backend: **Flask REST API** • ML Framework: **TensorFlow / Keras** • Computer Vision: ✓ **OpenCV** ✓ **MediaPipe Face Mesh** • Real-time webcam integration: **WebRTC**

Model Deployment	**Testing Environments**	• Local development (localhost with webcam testing) • Controlled user testing with sample SSL learners • Cross-browser testing (Chrome preferred for MediaPipe stability)	• Local (developer testing) • Staging (integration testing) • User testing (hearing-impaired learners) Testing includes: • Lip detection accuracy • Prediction correctness • Feedback clarity	• Local development server; API health monitored via /health endpoint polled every 3 s from the React frontend. Multi-run variance testing used to validate prediction consistency.	• Controlled testing with sample users • Different lighting condition testing • Different face angle testing • Browser-based testing • Real-time latency testing Testing includes: • face detection accuracy • gaze tracking correctness • attention state classification • alert trigger validation
	Deployment Platform	• Localhost deployment for development • Flask API deployment on cloud server • Frontend deployment using **Vercel**	• Localhost deployment for development • Flask API deployment on cloud server • Frontend deployment using **Vercel**	• Python FastAPI server on **localhost port 8000**; served via uvicorn ASGI. CORS middleware enabled for browser-to-localhost communication.	• Localhost deployment for development • Flask API deployment on cloud server • Frontend deployment using **Vercel**

		Backend deployment using **Railway**	Backend deployment using **Railway**		• Backend deployment using **Railway**
	Deployment Method	• Model serialized as .h5 / SavedModel • API endpoint for real-time gesture prediction • Continuous inference from webcam stream • Feedback loop integrated into UI	• Train model • Save version • Deploy API • Serve predictions	• Model loaded from .keras file at server startup. Frontend polls /predict every 500 ms with a 2-second WAV chunk (encoded in-browser); server responds in < 200 ms for near real-time emotion tracking.	• Model serialized as **.h5 / SavedModel** • REST API endpoint for real-time prediction • Continuous inference from webcam stream • Automated prediction feedback to frontend • Version-based model deployment
Future Enhancements	**Model Improvement:**	• Expand SSL dataset for better accuracy and regional variation coverage • Improve real-time feedback with confidence scoring and correction hints • Integrate 3D avatar-based gesture	• Expand Sinhala lip dataset • Improve phoneme-to-viseme mapping • ▯ Add adaptive feedback system	• Move beyond 4-quadrant categorisation to the full continuous valence-arousal space with nuanced emotion labels (e.g. nostalgia, triumph, serenity) • Offline-first architecture: package the CNN model with a local speech recognition engine	• improve face detection robustness under low light • advanced gaze estimation model • head pose-based attention scoring

		demonstration for better learning experience		(e.g. Whisper) to remove cloud dependencies • Expanded training data for improved accuracy across diverse music genres	• personalized focus behavior model • multi-face classroom support • emotion + attention fusion model